VITE SEO

Client Quotation Portal

Complete System Documentation

System Architecture | Security Design | Implementation Guide
Gap Analysis with Resolutions | Deployment Procedures

Document Title	Vite SEO Client Quotation Portal - Complete Documentation
Version	2.0 (Production Ready)
Original Date	February 3, 2026
Last Updated	February 03, 2026
Classification	Internal - Technical Architecture

STATUS: PRODUCTION READY

Table of Contents

PART I: OVERVIEW

1. Executive Summary
2. Project Goals and Non-Goals
3. System Overview

PART II: ARCHITECTURE

4. System Architecture
5. Technology Stack
6. Data Model
7. API Design

PART III: SECURITY

8. Security Architecture
9. Authentication Flow
10. Rate Limiting
11. Privacy Controls

PART IV: GAP ANALYSIS

12. Identified Gaps and Resolutions (14 Issues)

PART V: IMPLEMENTATION

13. User Flows
14. PDF Generation
15. Error Handling
16. Monitoring

PART VI: OPERATIONS

17. Performance
18. Disaster Recovery
19. Compliance
20. Deployment
21. Testing

APPENDICES

A. Environment Variables

B. Database Indexes

C. Audit Events

PART I: OVERVIEW

1. Executive Summary

This document describes the complete system architecture, design, and implementation guide for a secure client quotation portal for **Vite SEO Digital Advertising OPC**. The portal enables private, authenticated access for clients to view and download their quotations as PDFs.

The system uses a one-time passcode (OTP) authentication model where administrators generate access codes for clients. This approach eliminates password management while maintaining strong security controls.

Key Features:

- Private, OTP-gated client portals with no public browsing
- Custom-designed portal pages per client (developer-managed)
- PDF generation matching the web portal view exactly
- Complete audit trail of all access and actions
- Non-indexable by search engines with multiple privacy layers
- Rate limiting and abuse prevention

2. Project Goals and Non-Goals

Goals

- Provide a private portal per client to view quotations as a styled web page
- Allow clients to download the same quotation as a professionally formatted PDF
- Enable admins to maintain client-specific page designs in the GitHub repository
- Make the site non-indexable; expose no client content without authentication
- Maintain strong security: hashed OTPs, rate limiting, audit logs, secure cookies

Non-Goals

- Public browsing of quotations or client pages
- Self-service client account signup (clients use OTP access only)
- Arbitrary HTML/JS uploads from non-developers
- Real-time collaboration or commenting features

3. System Overview

The system follows a Next.js App Router architecture deployed on Vercel, with PostgreSQL for data persistence and optional Redis for rate limiting.

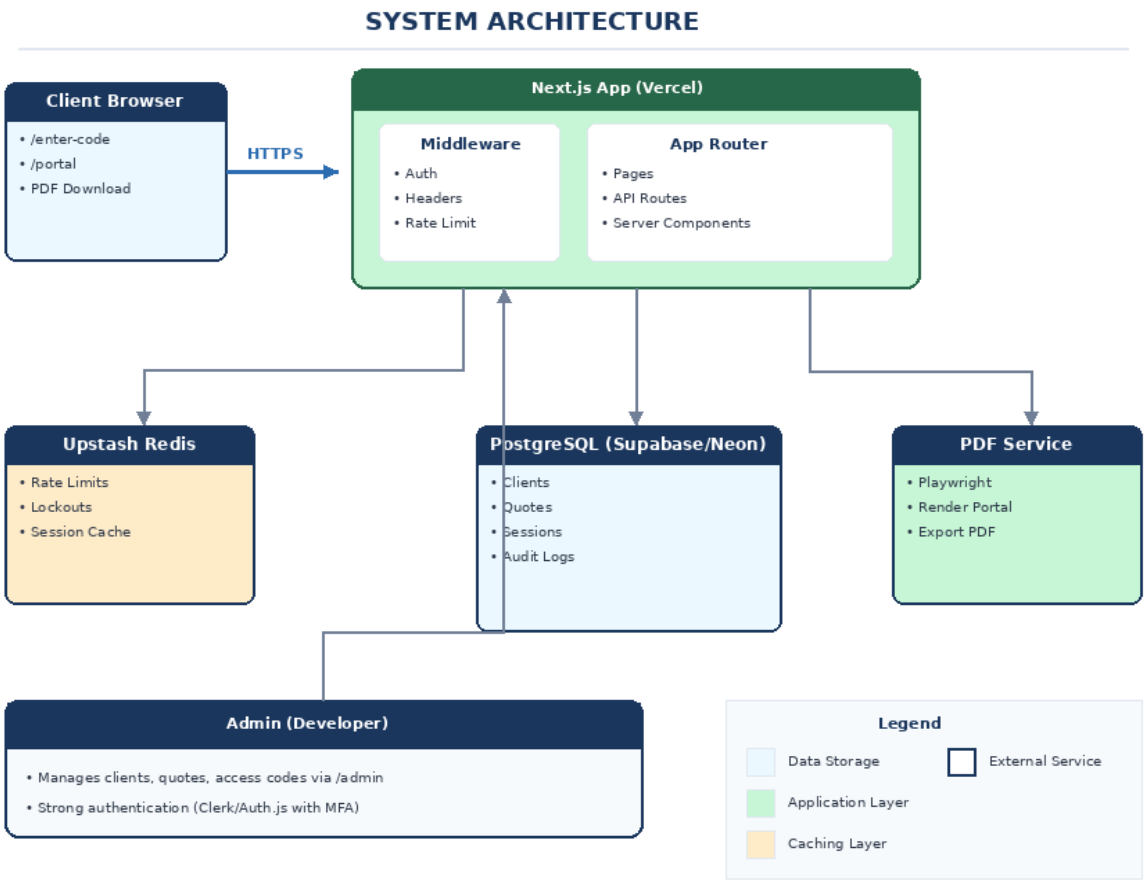
Key Design Decisions:

- **Server-first rendering:** React Server Components for security and performance
- **No client-side tokens:** Session managed via httpOnly cookies only
- **Immutable quotes:** Quotes become immutable after being sent
- **Defense in depth:** Multiple layers of security from edge to database

PART II: SYSTEM ARCHITECTURE

4. System Architecture

The following diagram shows the high-level system architecture including all major components and their interactions.



Component Responsibilities:

Component	Technology	Responsibility
Frontend	Next.js App Router + React	Server-rendered pages, client portal UI
Backend API	Next.js Route Handlers	REST API for auth, quotes, PDF generation
Database	PostgreSQL (Supabase/Neon)	Persistent storage for all data
Rate Limiting	Upstash Redis	Request throttling and lockouts

PDF Generation	Playwright	Headless browser PDF export

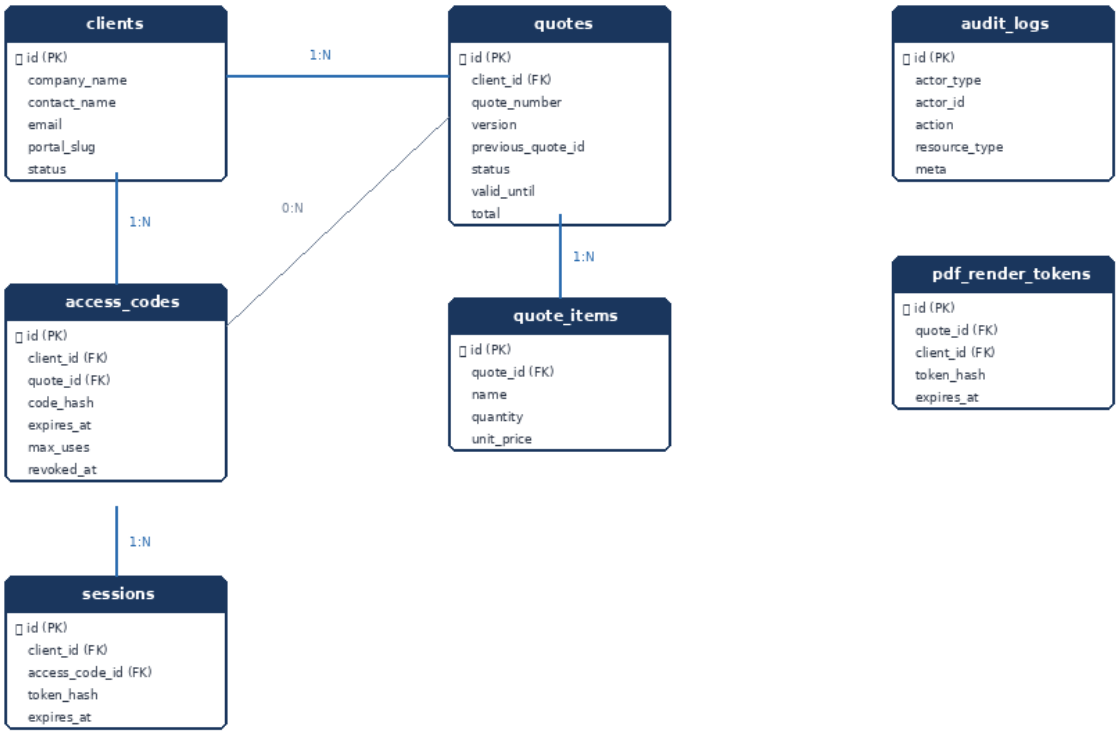
5. Technology Stack

Layer	Technology	Security Notes
Frontend + Backend	Next.js 14+ (App Router)	Server Components by default
Database	PostgreSQL	Row-level security; encryption at rest
ORM	Prisma	Type-safe queries; migrations
Admin Auth	Clerk or Auth.js v5	MFA support; secure sessions
Client Auth	Custom OTP + httpOnly cookies	No localStorage; strict cookies
Validation	Zod	Runtime type validation
Rate Limiting	Upstash Redis	Sliding window algorithm
PDF Generation	Playwright	Secure token authentication
UI Framework	Tailwind CSS + shadcn/ui	Consistent design system

6. Data Model

The data model is designed with security and auditability as primary concerns.

ENTITY RELATIONSHIP DIAGRAM



6.1 clients

Column	Type	Description
id	UUID	Primary identifier
company_name	VARCHAR(255)	Client company name
contact_name	VARCHAR(255)	Primary contact person
email	VARCHAR(255)	Contact email (encrypted)
portal_slug	VARCHAR(100)	URL-safe identifier for routing
status	ENUM	active | inactive | archived
timezone	VARCHAR(50)	Client timezone (IANA format)

6.2 quotes

Column	Type	Description
id	UUID	Primary identifier
client_id	UUID (FK)	Owning client
quote_number	VARCHAR(50)	Human-readable (Q-2026-0001)
version	INTEGER	Increments on revision
previous_quote_id	UUID (FK)	Reference to superseded quote
status	ENUM	draft|sent|viewed|accepted|rejected|expired|superseded
valid_until	TIMESTAMPTZ	Quote expiration date
total	DECIMAL(12,2)	Final amount including tax
content_hash	VARCHAR(64)	SHA-256 for cache invalidation

6.3 sessions

Column	Type	Description
id	UUID	Session identifier
client_id	UUID (FK)	Authenticated client
access_code_id	UUID (FK)	Access code used to create session

token_hash	VARCHAR(64)	SHA-256 hash of session token
ip_address	INET	Client IP at session creation
expires_at	TIMESTAMPTZ	Session expiration
revoked_at	TIMESTAMPTZ	If session was manually revoked

6.4 access_codes

Column	Type	Description
id	UUID	Primary identifier
client_id	UUID (FK)	Client this code grants access to
code_hash	VARCHAR(255)	Argon2id hash of the OTP
code_hint	VARCHAR(4)	Last 4 characters for admin reference
expires_at	TIMESTAMPTZ	Code expiration timestamp
max_uses	INTEGER	Maximum successful verifications
failed_attempts	INTEGER	Consecutive failed attempts
locked_until	TIMESTAMPTZ	Temporary lockout expiration
revoked_at	TIMESTAMPTZ	Manual revocation timestamp

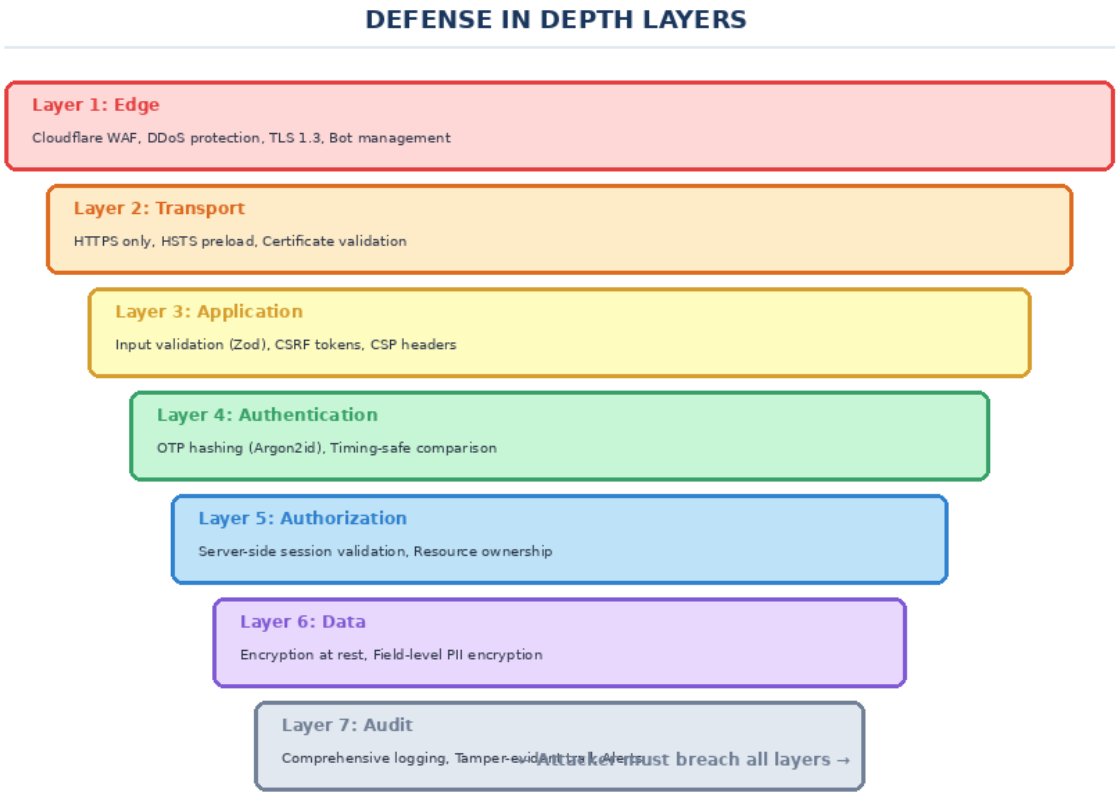
7. API Design

Endpoint	Method	Auth	Description
/api/auth/verify-code	POST	Public	Verify OTP and create session
/api/portal/me	GET	Client	Get current client profile
/api/portal/quote	GET	Client	Get client's current quote
/api/portal/pdf	POST	Client	Generate and download PDF
/api/admin/clients	GET/POST	Admin	List/create clients
/api/admin/quotes	GET/POST	Admin	List/create quotes
/api/admin/access-codes	GET/POST	Admin	Manage access codes
/api/health	GET	Public	System health check

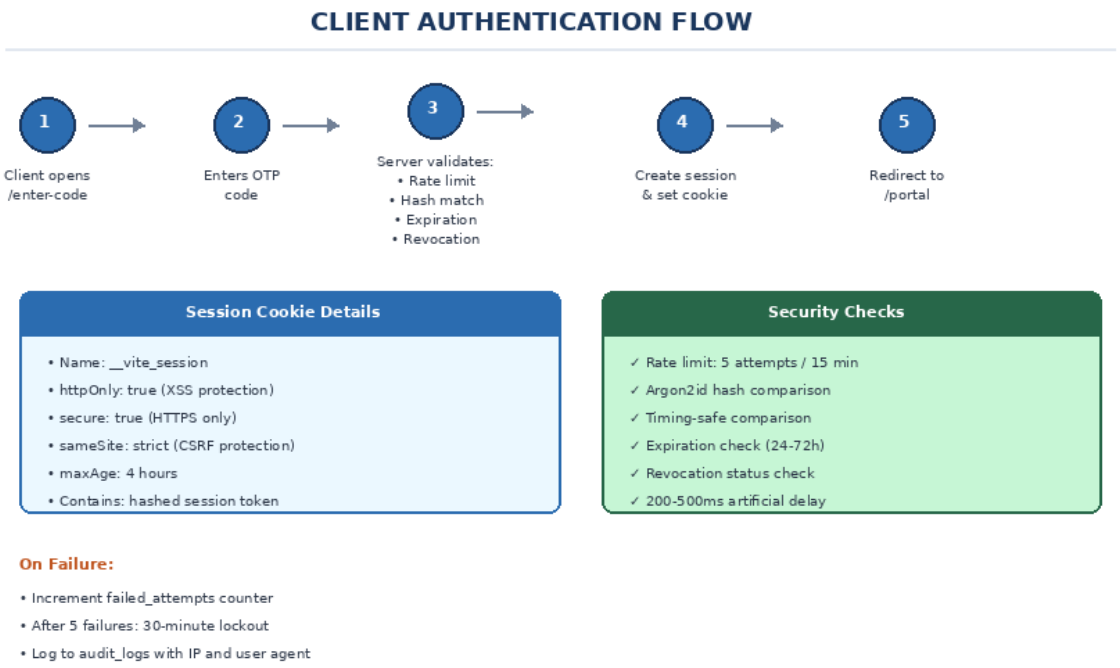
PART III: SECURITY

8. Security Architecture

The system implements multiple layers of security controls (defense in depth). If one layer is compromised, subsequent layers continue to protect the system.



9. Authentication Flow



OTP Specifications:

Property	Specification	Rationale
Format	8 alphanumeric characters	Excludes ambiguous chars (0/O, 1/I/l)
Entropy	~41 bits	Sufficient for rate-limited context
Storage	Argon2id hash	Memory-hard, GPU-resistant
Comparison	crypto.timingSafeEqual	Prevents timing attacks
Lifetime	24-72 hours	Balance security and usability

10. Rate Limiting and Abuse Prevention

Endpoint	Window	Limit	Lockout
/api/auth/verify-code	15 min	5 attempts	30 min after 5 fails
/api/portal/*	1 min	60 requests	429 response
/api/portal/pdf	1 hour	10 requests	429 response
/api/admin/*	1 min	120 requests	429 response

Progressive Lockout Strategy:

- Attempts 1-3: Normal verification with rate limiting
- Attempts 4-5: Require CAPTCHA (hCaptcha or Turnstile)
- Attempt 6+: Lock IP for 30 minutes, log to audit, alert admin
- Persistent abuse: 3+ lockouts/day triggers 24-hour edge block

11. Privacy and Bot Control

Layer 1: Authentication Gate

- Only /enter-code is public; reveals no client content
- /portal and APIs require valid session cookie
- Unauthenticated requests return 404 (not 401/403)

Layer 2: Anti-Indexing Directives

- robots.txt: User-agent: * Disallow: /
- HTTP Header: X-Robots-Tag: noindex, nofollow, noarchive
- Meta tag in HTML: noindex, nofollow
- No sitemap.xml published

PART IV: GAP ANALYSIS

14 Critical Gaps Identified and Resolved

A systematic review identified potential issues in the original design. Each gap has been addressed with concrete implementation guidance.

12. Identified Gaps and Resolutions

Gap	Severity	Issue	Resolution
GAP-001: Session Fixation	[CRITICAL]	No session rotation after OTP verification.	Generate new session ID after successful OTP. Invalidate pre-existing
GAP-002: Concurrent Sessions	[HIGH]	No limit on simultaneous sessions per client.	Max 2 concurrent sessions; new session invalidates oldest.
GAP-003: PDF Auth Gap	[CRITICAL]	Playwright auth mechanism not specified.	Short-lived PDF tokens (60s TTL, single-use) in request header.
GAP-004: Quote States	[MEDIUM]	No defined state machine for quotes.	7 states: DRAFT, SENT, VIEWED, ACCEPTED, REJECTED, EXPIRE
GAP-005: Timing Attack	[HIGH]	No timing-safe OTP comparison.	Use crypto.timingSafeEqual + 200-500ms artificial delay.
GAP-006: Code Revocation	[MEDIUM]	No explicit OTP revocation mechanism.	Add revoked_at, revoked_by columns + revoke endpoint.
GAP-007: Session/Code Link	[MEDIUM]	Sessions live independently of OTP expiration.	Link sessions to access codes; invalidate on code expiry.
GAP-008: CSRF Protection	[HIGH]	No CSRF tokens for state-changing operations.	Double-submit cookie pattern + SameSite=Strict.
GAP-009: Error Strategy	[LOW]	No standard error response format.	Schema: { error: { code, message, requestId } }.
GAP-010: Quote Versioning	[MEDIUM]	No handling for modifications after viewing.	Quotes immutable after SENT; modifications create new version.
GAP-011: Timezone Handling	[LOW]	Dates without timezone specification.	Store UTC; display in client's local timezone.
GAP-012: PDF Caching	[LOW]	No caching strategy for generated PDFs.	Cache in S3/R2 keyed by quote_id:content_hash.
GAP-013: Health Checks	[MEDIUM]	No health monitoring endpoints.	Add /api/health with dependency checks.
GAP-014: Data Retention	[MEDIUM]	No data retention policy specified.	Define retention: audit logs 7yr, inactive clients 3yr.

PART V: IMPLEMENTATION

13. User Flows

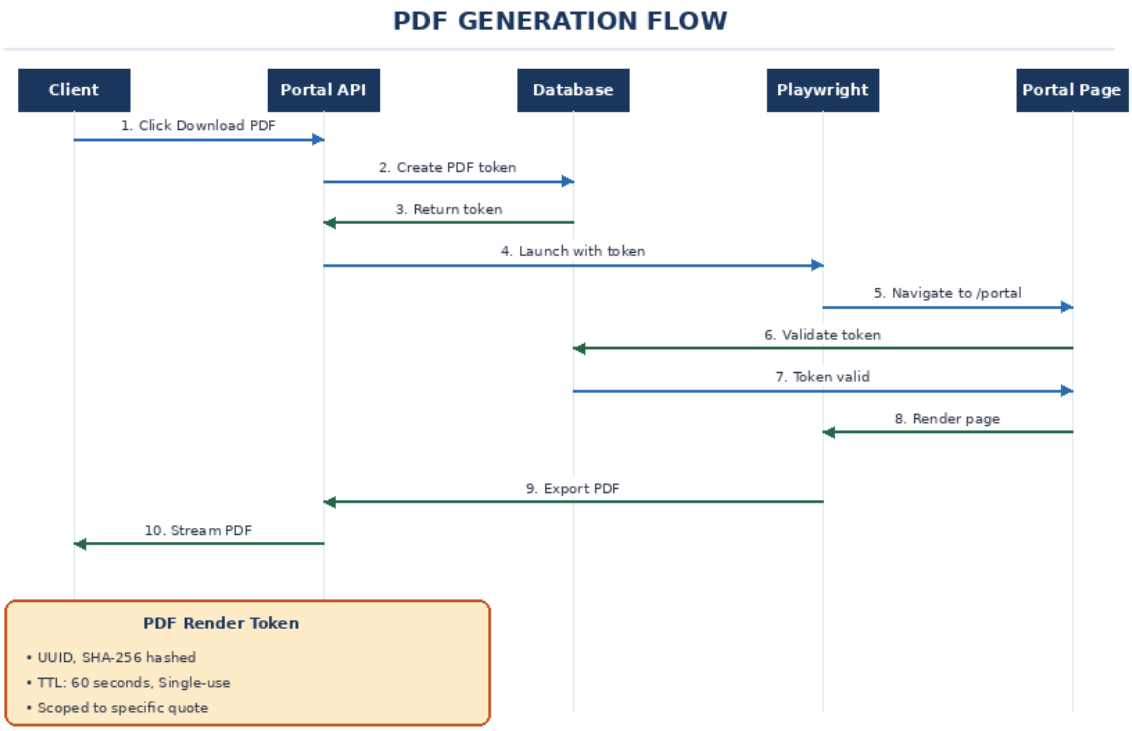
Admin Flow:

1. Admin signs in to /admin using Clerk (with MFA)
2. Admin creates or selects a Client record
3. Admin creates a Quote with line items, totals, validity dates
4. Admin generates an Access Code (OTP) for the client
5. Admin shares the portal link and OTP through trusted channel

Client Flow:

1. Client opens shared link (e.g., /enter-code)
2. Client enters the OTP/access code
3. System validates and creates session with secure cookie
4. Client views quotation on personalized portal page
5. Client clicks Download PDF to get formatted document

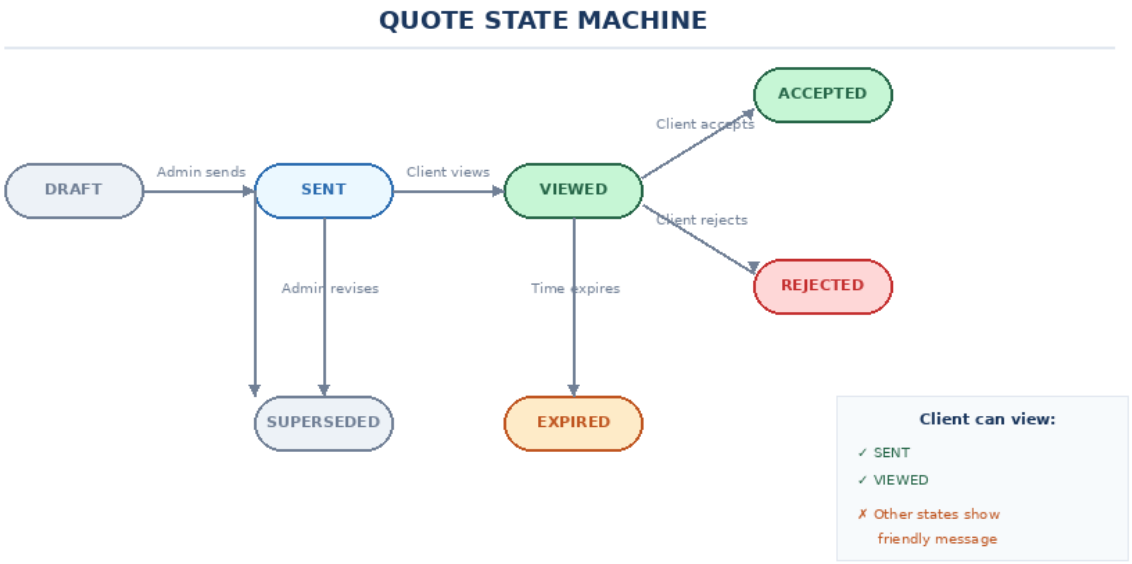
14. PDF Generation



PDF Caching Strategy:

- Cache Key: {quote_id}:{content_hash}
- Storage: Cloudflare R2 or AWS S3 with encryption
- TTL: 24 hours or until quote status changes
- Headers: Cache-Control: private, no-store

15. Quote State Machine



16. Error Handling

Code	HTTP	Description
INVALID_OTP	401	OTP incorrect, expired, or revoked
RATE_LIMITED	429	Too many attempts in time window
SESSION_EXPIRED	401	Session timed out or revoked
QUOTE_NOT_FOUND	404	Quote doesn't exist or not authorized
QUOTE_EXPIRED	410	Quote past validity date
PDF_GENERATION_FAILED	500	PDF service error

PART VI: OPERATIONS

17. Performance Targets

Operation	Target (p95)	Maximum
OTP verification	200ms	500ms
Portal page load	300ms	800ms
Quote data fetch	100ms	300ms
PDF generation	5 seconds	15 seconds

18. Disaster Recovery

Scenario	RTO	RPO	Procedure
Database corruption	1 hour	< 1 hour	Restore from backup, replay WAL
Region failure	4 hours	< 1 hour	Failover to secondary region
Security breach	Immediate	N/A	Revoke sessions, rotate secrets

19. Data Retention Policy

Data Type	Retention	After Retention
Active clients	Indefinite	N/A
Inactive clients	3 years	Archive then delete
Quotes	7 years	Archive to cold storage
Audit logs	7 years	Archive to cold storage
Expired sessions	7 days	Hard delete
Expired access codes	90 days	Hard delete

20. Deployment Checklist

Category	Item
Security	All environment variables configured
Security	Database connection uses SSL
Security	Admin authentication with MFA
Security	Rate limiting tested
Security	CSRF protection enabled
Privacy	robots.txt returns Disallow: /
Privacy	X-Robots-Tag header present
Functionality	OTP verification flow tested
Functionality	PDF generation working
Operations	Health check endpoint responding
Operations	Error monitoring configured
Operations	Database backups verified

21. Testing Strategy

Component	Test Type	Coverage Target
API Routes	Unit + Integration	> 80%
OTP Logic	Unit	100%
Session Management	Unit + Integration	> 90%
Authorization	Integration	All paths
PDF Generation	E2E	Happy path + errors

APPENDICES

Appendix A: Environment Variables

Variable	Required	Description
DATABASE_URL	Yes	PostgreSQL connection string
UPSTASH_REDIS_REST_URL	Yes	Redis URL for rate limiting
UPSTASH_REDIS_REST_TOKEN	Yes	Redis auth token
CLERK_SECRET_KEY	Yes*	Clerk backend secret
APP_URL	Yes	Full application URL
SESSION_SECRET	Yes	Session token signing secret
ENCRYPTION_KEY	Yes	PII field encryption key
SENTRY_DSN	Recommended	Error tracking DSN

Appendix B: Database Indexes

Table	Index	Purpose
access_codes	idx_codes_lookup (code_hash, expires_at)	OTP verification
sessions	idx_sessions_token (token_hash)	Session lookup
sessions	idx_sessions_client (client_id, expires_at)	Concurrent check
quotes	idx_quotes_client (client_id, status)	Client quotes
audit_logs	idx_audit_actor (actor_type, actor_id)	Actor history

Appendix C: Audit Log Events

Event	Actor	Data Captured
OTP_VERIFIED	client	clientId, accessCodeId, ip
OTP_FAILED	anonymous	codeHint, ip, reason

SESSION_CREATED	system	sessionId, clientId, ip
SESSION_REVOKED	admin/system	sessionId, reason
QUOTE_VIEWED	client	quoteId, sessionId
PDF_DOWNLOADED	client	quoteId, sessionId
ACCESS_CODE_CREATED	admin	accessCodeId, clientId
RATE_LIMIT_HIT	system	ip, endpoint

— End of Document —

Last updated: February 2026 | Security audit recommended quarterly